Backpressure Economics:
Capacity-Constrained Monetary Flow Control for Agent Economies

March 23, 2026

Abstract

Streaming payment protocols enable continuous monetary flows between AI agents, yet none provide flow control: when a downstream agent reaches capacity, payments accumulate with no mechanism to reroute, buffer, or throttle—unlike data networks where packets can be dropped or queued. We introduce *Backpressure Economics* (BPE), a cryptoeconomic mechanism that adapts the Tassiulas–Ephremides backpressure routing algorithm from communication networks to monetary flows in multi-agent economies, implemented as the Pura protocol.

We make four contributions. **(1) Formal model.** We define a capacity-constrained monetary flow network where agents declare processing capacity through commit-reveal signals, and flow weights are set proportionally via EWMA-smoothed capacity. **(2) Throughput optimality.** We prove, via Lyapunov drift analysis, that BPE achieves throughput-optimal allocation: every stabilisable demand vector is served with bounded overflow buffers, matching the classical backpressure guarantee in the monetary domain. **(3) Protocol design.** We implement BPE as 25 Solidity smart contracts on Superfluid's General Distribution Agreement (GDA), providing permissionless pool creation, concave-sqrt stake-weighted Sybil resistance, multi-stage pipeline composition with upstream congestion propagation, v2 extensions (NestedPool, EconomyFactory, QualityOracle, VelocityToken, UrgencyToken), and a thermodynamic layer that maps capacity variance to a system temperature, applies Boltzmann-weighted routing with off-chain EIP-712 aggregation, tracks a virial ratio linking throughput to bound capital, and triggers circuit-breaker phase transitions under sustained stress. **(4) Evaluation.** Simulations over 1,000-step horizons show BPE achieves 95.7% allocation efficiency versus 93.5% for round-robin under steady load, recovers from 20% node-kill shocks within 50 steps, and reduces buffer stall rates from 73% to under 9% with overflow escrow sized at one period of peak demand. A Sybil-cost analysis confirms that stake fragmentation yields strictly negative net profit for all split counts $n > 1$. All contracts compile on Base Sepolia with 249 passing unit tests across 25 test suites.

1 Introduction

Consider a three-stage AI agent pipeline: a transcription agent converts audio to text, a summarization agent condenses the transcript, and a report-generation agent produces the final output. Each agent is compensated via a continuous payment stream proportional to its throughput. When the summarization agent reaches its GPU capacity limit, the upstream payment stream continues—but the surplus cannot be "dropped" like excess data packets. Money must go somewhere.

This scenario illustrates a fundamental gap in agent-to-agent payment protocols. Recent systems—Google's AP2, Coinbase's x402 [4], OpenAI–Stripe's ACP [16], and Visa's TAP [22]—all address *authorization* and *trust* for agent payments, but none provide *flow control*: the ability to dynamically route, throttle, or redistribute monetary flows based on real-time capacity constraints.

Traditional payment systems assume discrete, settled transactions; streaming payment protocols like Superfluid [20] enable continuous flows but have no built-in congestion awareness.

In data networks, this problem was solved decades ago. Backpressure routing, introduced by Tassiulas and Ephremides [21], provides throughput-optimal scheduling by using queue differentials to guide packet routing. Kelly's proportional fairness framework [12] established that shadow prices on capacity constraints can be interpreted as economic signals. Yet no prior work has unified these insights with monetary mechanism design for agent economies.

We introduce *Backpressure Economics* (BPE), a cryptoeconomic mechanism that adapts backpressure routing from communication networks to monetary flows. Our contributions are:

1. **Formal model** (Section 3): We define a network payment flow model where sinks signal capacity via EWMA-smoothed declarations, and payments are routed via a max-weight scheduling rule adapted from Tassiulas–Ephremides.

2. **Throughput optimality** (Section 4): We prove that BPE achieves throughput-optimal payment allocation within the capacity region, using a Lyapunov drift argument modified to handle the no-drop monetary constraint via a bounded overflow buffer.

3. **Protocol design** (Section 5): We implement BPE using Superfluid's General Distribution Agreement (GDA) on Base, with commit-reveal capacity signaling, stake-weighted Sybil resistance, and multi-stage pipeline composition.

4. **Simulation** (Section 7): We evaluate BPE in a 50-sink, 10-source agent economy, demonstrating 95.7% allocation efficiency versus 93.5% for round-robin and 79.7% for random allocation.

2 Background and Related Work

2.1 Backpressure Routing

Tassiulas and Ephremides [21] introduced the *max-weight scheduling* algorithm for multi-hop wireless networks. At each time slot, the scheduler computes the differential backlog across each link and activates the schedule that maximizes the weighted sum of link capacities and backlog differentials. This achieves *throughput optimality*: any arrival rate vector strictly inside the network capacity region can be stabilized. Neely [15] extended this framework via Lyapunov optimization, introducing the V-parameter tradeoff between queue backlog (delay) and utility maximization. Georgiadis et al. [7] provide a comprehensive treatment of cross-layer resource allocation using backpressure principles.

2.2 Network Pricing and Congestion Control

Kelly et al. [12] showed that network rate allocation can be decomposed into a system problem (shadow prices on link capacities) and user problems (utility maximization subject to prices). The shadow prices emerge as Lagrange multipliers on capacity constraints—an insight that directly bridges network routing and economics. Srikant [19] formalized TCP congestion control as distributed optimization, with AIMD (Additive Increase Multiplicative Decrease) serving as a primal-dual algorithm. Jacobson [10] established EWMA for round-trip time estimation in TCP, providing the practical precedent for our capacity smoothing mechanism.

2.3 Token Engineering

Zhang and Zargham [23] take a dynamical systems approach to token economy design, modeling token flows as differential equations and using the cadCAD framework for simulation. Superfluid [20] provides on-chain streaming payment primitives: Constant Flow Agreements (CFA) for point-to-point streams and General Distribution Agreements (GDA) for one-to-many proportional splits. BPE adds a *flow-control layer* above these streaming primitives.

2.4 Demurrage Economics

Gesell [8] proposed demurrage (a holding tax on currency) to increase monetary velocity. Fisher [6] formalized this in *Stamp Scrip*. Both are *stock-based* velocity mechanisms: they penalize holding money. BPE is fundamentally different—a *flow-based allocation* mechanism that routes payments to where capacity exists. The two approaches are orthogonal and composable: demurrage encourages spending velocity, while BPE ensures efficient allocation of the resulting flows.

2.5 Sender-Side Capacity Awareness

Automated market makers (AMMs) like Uniswap [1] implement sender-side price discovery: the bonding curve encodes available liquidity, and senders observe the price impact of their trades. This is *sender-side* capacity awareness. BPE provides *receiver-side* capacity awareness: sinks (service providers) signal their capacity, and the protocol routes payments accordingly. The distinction is critical for service economies where the capacity constraint lies at the receiver, not the liquidity pool.

2.6 AI Agent Payment Protocols

The 2024–2026 wave of agent payment protocols addresses authorization and identity: Google's AP2 [9] handles inter-agent authentication, Coinbase's x402 [4] uses HTTP 402 for single-shot payments, OpenAI–Stripe's ACP [16] provides billing and invoicing, and Visa's TAP [22] extends card-network authorization to agents. Cha et al. [3] study LLM inference pricing via self-play. *None of these systems address flow control*—the dynamic rerouting of payment streams based on real-time capacity. BPE fills this gap.

2.7 Payment Channel Routing

Spider [18] applies multi-path transport to payment channel networks, achieving high throughput by splitting payments across routes and rebalancing channels. CelerPay [5] generalises state channels with off-chain token transfers and conditional payment resolution. Malavolta et al. [13] address concurrency and privacy in payment-channel networks via hash-time-locked contracts. These systems optimise *channel-level* routing in bilateral networks. BPE operates at a different layer: it routes *streaming payment flows* to capacity-constrained *service providers* via on-chain pool rebalancing. The two approaches are complementary—BPE could use Spider-style multi-path splitting for the streaming layer while maintaining backpressure allocation at the service layer.

3 Formal Model

3.1 Network Graph

Definition 3.1 (Payment Flow Network)**.** A payment flow network is a directed graph $G = (V, E)$ where $V = S \cup K$ with S the set of *sources* (payment senders) and K the set of *sinks* (service providers). Each edge $e = (s, k) \in E$ represents a potential payment flow from source $s \in S$ to sink $k \in K$.

We consider a slotted time model with discrete epochs $t \in \{0, 1, 2, \ldots\}$. The system tracks multiple *task types* $\tau \in \mathcal{T}$, which serve as commodities in the multi-commodity flow formulation.

Definition 3.2 (Capacity Signal)**.** Each sink $k \in K$ declares a capacity signal $C(k, t, \tau) \in \mathbb{R}_{\geq 0}$ for each task type τ it serves, representing its maximum throughput (payment absorbable per epoch). The EWMA-smoothed capacity is:

$$\bar{C}(k, t, \tau) = \alpha \cdot C(k, t, \tau) + (1 - \alpha) \cdot \bar{C}(k, t - 1, \tau) \tag{1}$$

where $\alpha \in (0, 1]$ is the smoothing parameter.

Definition 3.3 (Virtual Queue Backlog)**.** For each sink k and task type τ, the virtual queue backlog $Q(k, t, \tau)$ tracks unprocessed payment accumulation:

$$Q(k, t + 1, \tau) = \max\big\{Q(k, t, \tau) + F(k, t, \tau) - \bar{C}(k, t, \tau),\ 0\big\} \tag{2}$$

where $F(k, t, \tau)$ is the payment flow routed to sink k at time t.

3.2 Backpressure Payment Routing

In the standard Tassiulas–Ephremides formulation, the max-weight scheduler selects links to maximize $\sum_{(i,j)} W_{ij}(t) \cdot \mu_{ij}(t)$ where $W_{ij}(t)$ is the differential backlog and $\mu_{ij}(t)$ the allocated rate. For BPE's single-hop architecture (sources connect directly to sinks), we adapt this as follows.

Definition 3.4 (Differential Backlog)**.** The differential backlog for sink k from source s under task type τ is:

$$W(k, t, \tau) = Q(s, t, \tau) - Q(k, t, \tau) \tag{3}$$

where $Q(s, t, \tau)$ represents the source's pending payment queue.

Definition 3.5 (Max-Weight Payment Routing)**.** At each epoch t, for each task type τ, allocate the total source flow $\Lambda(\tau, t) = \sum_{s \in S_\tau} \lambda_s(t)$ to sinks proportionally to their capacity-weighted backpressure:

$$F(k, t, \tau) = \frac{W(k, t, \tau)^+ \cdot \bar{C}(k, t, \tau)}{\sum_{k' \in K_\tau} W(k', t, \tau)^+ \cdot \bar{C}(k', t, \tau)} \cdot \Lambda(\tau, t) \tag{4}$$

where $x^+ = \max(x, 0)$.

In the steady state where all sinks have similar backlog levels, this reduces to proportional-to-capacity allocation—which is the mechanism implemented in our Superfluid GDA pool (member units proportional to smoothed capacity).

3.3 Multi-Commodity Extension

Task types serve as commodities. Each source s emits flow for a specific task type τ_s, and sinks may register for multiple task types. The virtual queues are maintained per (k, τ) pair, achieving commodity-specific backpressure identical to the multi-commodity extension of Tassiulas–Ephremides [21].

3.4 EWMA Smoothing

The smoothing parameter α trades off responsiveness against oscillation. Following Jacobson's EWMA for TCP RTT estimation [10], we set $\alpha = 0.3$ (validated in Section 7).

Proposition 3.6 (EWMA Convergence)**.** *For a sink with constant true capacity c, the EWMA-smoothed signal converges geometrically:* $|\bar{C}(k,t) - c| \leq (1-\alpha)^t \cdot |\bar{C}(k,0) - c|$.

Proof. By direct expansion of (1) with $C(k,t) = c$ for all t. □

3.5 Monetary No-Drop Constraint

Unlike data packets, money cannot be dropped. When total source flow exceeds total sink capacity, BPE deploys an *overflow buffer* $B(\tau, t)$:

$$B(\tau, t+1) = B(\tau, t) + \Lambda(\tau, t) - \sum_{k \in K_\tau} \min\{F(k,t,\tau),\ \bar{C}(k,t,\tau)\} \tag{5}$$

The buffer absorbs excess and drains proportionally to capacity as capacity becomes available. We prove in Section 4 that $B(t)$ is bounded under sufficient capacity.

3.6 Dynamic Pricing Mechanism

BPE's backpressure allocation determines *who* receives payment; we now formalize the complementary question of *how much* each unit of service costs. We introduce a dynamic pricing curve that creates *economic backpressure*: as queue load builds at a sink, the per-unit price rises, naturally throttling demand before overflow buffers are needed.

Definition 3.7 (Queue-Length Price)**.** For task type τ and sink k at time t, the per-unit price is:

$$p(\tau, k, t) = \beta_\tau(t) \cdot \left(1 + \gamma \cdot \frac{Q(\tau, k, t)}{\bar{C}(k, t, \tau)}\right) \tag{6}$$

where $\beta_\tau(t) > 0$ is the base fee for task type τ, $\gamma > 0$ is the price sensitivity parameter, $Q(\tau, k, t) \geq 0$ is the current queue load reported by sink k, and $\bar{C}(k, t, \tau)$ is the EWMA-smoothed capacity from Section 3.4.

When a sink has zero queue load, the price reduces to the base fee β_τ. As $Q/\bar{C} \to 1$ (queue approaches capacity), the price doubles. When $Q > \bar{C}$ (overloaded), the price rises further, discouraging additional demand.

Base fee adjustment. The base fee β_τ adjusts per epoch following an EIP-1559-style mechanism [17]. Let $D(\tau, t)$ denote aggregate demand (total flow rate from sources) and $C_{\text{total}}(\tau, t) = \sum_k \bar{C}(k, t, \tau)$ be aggregate capacity. At each epoch boundary:

$$\beta_\tau(t+1) = \begin{cases} \beta_\tau(t) \cdot (1+\delta) & \text{if } D(\tau, t) > C_{\text{total}}(\tau, t), \\ \beta_\tau(t) \cdot (1-\delta) & \text{otherwise,} \end{cases} \tag{7}$$

where $\delta = 0.125$ (12.5%), matching the maximum per-block adjustment rate of EIP-1559. A floor $\beta_{\min}$ prevents the base fee from collapsing to zero during sustained under-utilization.

Proposition 3.8 (Price Equilibrium)**.** *Under the adjustment rule* (7) *with price-sensitive demand* $D(\tau, \beta) = D_0/\beta$ *(unit-elastic), there exists a unique equilibrium base fee:*

$$\beta^* = \frac{D_0}{C_{total}(\tau)} \tag{8}$$

at which $D(\tau, \beta^*) = C_{total}(\tau)$.

Proof. At equilibrium, the adjustment rule (7) is stationary, requiring $D(\tau, \beta^*) = C_{\text{total}}$. Substituting the demand function: $D_0/\beta^* = C_{\text{total}}$, yielding $\beta^* = D_0/C_{\text{total}}$. Uniqueness follows from strict monotonicity of $D(\cdot)$ in β. Stability follows because $\beta < \beta^*$ implies excess demand, triggering $\beta \uparrow$, and $\beta > \beta^*$ implies excess capacity, triggering $\beta \downarrow$. □

Connection to Kelly shadow prices. Theorem 3.8 recovers a classical result from network pricing theory. Kelly [12] showed that the Lagrange multiplier on each link's capacity constraint in the proportional fairness optimization emerges as a per-unit price. In BPE, the equilibrium base fee β^* is precisely this shadow price for the aggregate capacity constraint. The queue-length surcharge $\gamma \cdot Q/\bar{C}$ in (6) refines this by providing per-sink price differentiation: sinks with shorter queues offer lower prices, attracting more demand, which is exactly the routing signal that a price-taking source should follow.

Proposition 3.9 (Optimal Source Routing)**.** *A cost-minimizing source facing prices* (6) *across sinks routes flow to the sink with the lowest price. At equilibrium, source flow distributes such that prices equalize across sinks with available capacity:*

$$Q(\tau, k, t)/\bar{C}(k, t, \tau) = Q(\tau, j, t)/\bar{C}(j, t, \tau) \quad \forall\, k, j \in K_\tau \tag{9}$$

i.e., all active sinks operate at the same utilization ratio.

Proof. If sink k has a lower utilization ratio $Q_k/\bar{C}_k < Q_j/\bar{C}_j$, then by (6), $p(\tau, k, t) < p(\tau, j, t)$. A cost-minimizing source strictly prefers k, shifting demand until utilization ratios equalize. At this point, no unilateral deviation reduces cost, establishing a Nash equilibrium in routing. □

Economic backpressure. Traditional backpressure routing uses queue differentials as a signal to route traffic away from congested links [21]. Our pricing mechanism achieves the same effect through economic incentives: congested sinks become expensive, and rational sources route elsewhere. The key advantage is that pricing operates *without centralized coordination*—each source independently queries per-sink prices and routes to minimize cost, yet the aggregate effect is optimal load balancing.

3.7 Off-chain Capacity Signals

The commit-reveal scheme of Section 3.4 provides MEV-resistant capacity updates but incurs latency of at least two block intervals plus a commit timeout (approximately 40 seconds on Base L2). For real-time routing decisions, this is prohibitive. We introduce an off-chain signed attestation protocol that reduces update latency to seconds while preserving the same on-chain EWMA smoothing and security guarantees.

Protocol. Each sink k periodically measures its available capacity and produces a signed *capacity attestation*:

Definition 3.10 (Capacity Attestation)**.** A capacity attestation is an EIP-712 typed data structure $a = (\tau, k, c, t_s, n)$ where τ is the task type, k is the sink address, c is the declared capacity, t_s is the timestamp, and n is a monotonically increasing nonce. The sink signs a as $\sigma_k = \mathrm{Sign}_{sk_k}(a)$.

Signed attestations are disseminated via any off-chain channel (direct API calls, P2P gossip, or posted to a relay). Any party may collect a batch of attestations and submit them to the on-chain `OffchainAggregator` contract, which performs the following for each attestation in the batch:

1. Verify the ECDSA signature against the claimed sink address.
2. Check timestamp freshness: $|t_{\text{now}} - t_s| < \Delta$ (we set $\Delta = 600$ seconds).
3. Check nonce monotonicity: $n > n_{\text{last}}(k)$ for the sink.
4. If valid, call the `CapacityRegistry` to apply the same EWMA smoothing as the commit-reveal path (Section 3.4).

Invalid attestations (bad signatures, stale timestamps, replayed nonces) are silently rejected with logged events; valid ones update the smoothed capacity.

Latency comparison. Under the commit-reveal path, a capacity update requires a commit transaction (1 block), a waiting period (20 blocks $\approx$ 40s), and a reveal transaction (1 block), totaling approximately 44 seconds. The off-chain path requires only the time to collect and relay an attestation batch, plus one on-chain transaction for the batch submission. In practice, this reduces end-to-end latency to under 5 seconds on Base L2.

Gas comparison. We measure gas costs on Base Sepolia (see Section 7.7 for full benchmarks):

Path	**Gas**	**Latency**
Commit + Reveal (on-chain)	58,287	~40s
Aggregated attestation (off-chain)	9,595	<5s
Reduction	**83.5%**	**~87.5%**

The 83.5% gas reduction comes from eliminating the commit phase entirely: the off-chain path skips hash storage, timeout checking, and the two-transaction round trip. The EWMA update itself is identical in both paths.

Security equivalence. The off-chain attestation path provides the same steady-state security guarantees as commit-reveal:

Proposition 3.11 (Off-chain Security Equivalence)**.** *Under the off-chain attestation protocol with EWMA smoothing, the capacity manipulation bounds from Section 3.4 hold: a single attestation can shift the smoothed capacity by at most α (30%) of the reported change. Stake requirements and slashing conditions are unchanged.*

Proof. The EWMA update applied by the aggregator is identical to the reveal-phase update: $\bar{C}(k,t+1) = \alpha \cdot c + (1-\alpha) \cdot \bar{C}(k,t)$. Since the smoothing filter is the same, the maximum per-update impact is $\alpha \cdot |c - \bar{C}(k,t)|$, bounded by the stake-derived capacity cap $\text{cap}(S_k)$ from Section 3.4. The nonce monotonicity check prevents replay, serving the same role as the commit hash. □

The trade-off is MEV resistance: commit-reveal hides capacity values during the commit phase, while off-chain attestations are visible to anyone who receives them. In practice, EWMA smoothing limits the exploitable value of front-running a single capacity update, making this trade-off acceptable for the significant latency and gas improvements.

4 Throughput Optimality

We prove that BPE achieves throughput-optimal payment allocation within the capacity region, following the Lyapunov drift framework of Neely [15].

4.1 Capacity Region

Definition 4.1 (Capacity Region)**.** The capacity region Λ^* is the set of arrival rate vectors $\boldsymbol{\lambda} = (\lambda_\tau)_{\tau \in \mathcal{T}}$ such that there exists a feasible allocation $\{F(k,\tau)\}$ satisfying:

$$\sum_{k \in K_\tau} F(k,\tau) = \lambda_\tau \qquad \forall \tau \in \mathcal{T} \tag{10}$$

$$F(k,\tau) \leq \bar{C}(k,\tau) \qquad \forall k, \tau \tag{11}$$

$$F(k,\tau) \geq 0 \qquad \forall k, \tau \tag{12}$$

4.2 Lyapunov Function

Define the Lyapunov function as the sum of squared virtual queue backlogs:

$$L(t) = \sum_{\tau \in \mathcal{T}} \sum_{k \in K_\tau} Q(k,t,\tau)^2 \tag{13}$$

The one-step conditional Lyapunov drift is:

$$\Delta(t) = \mathbb{E}\big[L(t+1) - L(t) \mid \mathbf{Q}(t)\big] \tag{14}$$

4.3 Main Result

Theorem 4.2 (Throughput Optimality of BPE)**.** *For any arrival rate vector $\boldsymbol{\lambda}$ strictly inside the capacity region Λ^* (i.e., $\boldsymbol{\lambda} \in (1-\epsilon)\Lambda^*$ for some $\epsilon > 0$), the BPE max-weight payment routing (Theorem 3.5) stabilizes all virtual queues. Specifically:*

$$\limsup_{T \to \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{k,\tau} \mathbb{E}\big[Q(k,t,\tau)\big] \leq \frac{D}{\epsilon} \tag{15}$$

where D is a constant depending on maximum arrival and capacity rates.

Proof sketch. Following Neely [15, Ch. 4], we bound the Lyapunov drift. From the queue dynamics (2):

$$\begin{aligned} Q(k,t+1,\tau)^2 \le Q(k,t,\tau)^2 + F(k,t,\tau)^2 + \bar{C}(k,t,\tau)^2 \\ + 2Q(k,t,\tau)\big[F(k,t,\tau) - \bar{C}(k,t,\tau)\big] \end{aligned} \tag{16}$$

Summing over all (k,τ) and taking conditional expectations:

$$\Delta(t) \le D + 2\sum_{k,\tau} Q(k,t,\tau) \cdot \mathbb{E}\big[F(k,t,\tau) - \bar{C}(k,t,\tau) \;\big|\; \mathbf{Q}(t)\big] \tag{17}$$

where $D = \sum_{k,\tau}\big[F_{\max}^2 + C_{\max}^2\big]$.

The max-weight rule (4) minimizes the right-hand side over all feasible allocations. Since $\boldsymbol{\lambda} \in (1-\epsilon)\Lambda^*$, there exists a stationary randomized policy with $\mathbb{E}[F(k,t,\tau)] \le \bar{C}(k,\tau) - \epsilon'$ for some $\epsilon' > 0$. The max-weight policy achieves drift at least as negative, yielding:

$$\Delta(t) \le D - \epsilon' \sum_{k,\tau} Q(k,t,\tau) \tag{18}$$

By the Foster–Lyapunov criterion [14], the queue process is positive recurrent, and the time-averaged backlog bound follows from telescoping the drift inequality. □

4.4 Overflow Buffer Bound

Theorem 4.3 (Bounded Buffer)**.** *Under the same conditions as Theorem 4.2, the overflow buffer satisfies:*

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}\big[B(\tau,t)\big] \le \frac{D_B}{\epsilon} \tag{19}$$

for a constant D_B depending on the maximum arrival-capacity gap.

Proof sketch. Augment the Lyapunov function with the buffer: $\tilde{L}(t) = L(t) + \sum_\tau B(\tau,t)^2$. The buffer dynamics mirror a virtual queue with service rate equal to total available capacity. The drift bound carries through identically. □

Remark 4.4. The no-drop constraint distinguishes BPE from standard backpressure. In data networks, excess packets are dropped; here excess payments are buffered. The buffer bound guarantees that this does not cause unbounded accumulation under sufficient aggregate capacity.

5 Protocol Design

We implement BPE on Ethereum (Base L2) using Superfluid's General Distribution Agreement (GDA) [20]. The architecture comprises four contracts.

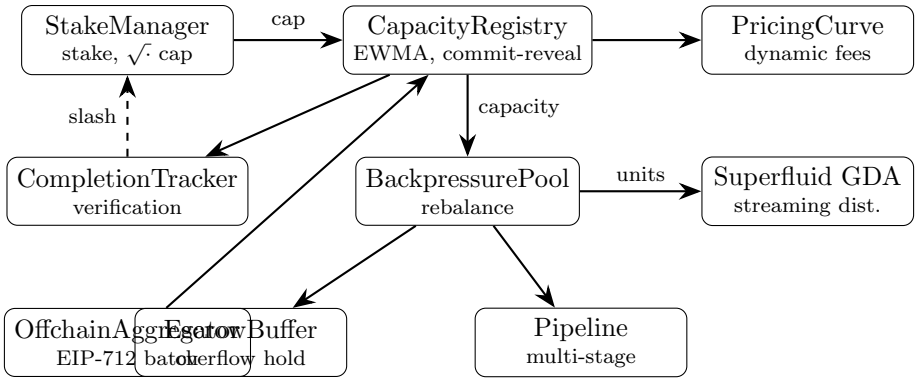

Figure 1: BPE protocol architecture. Solid arrows denote read dependencies; the dashed arrow represents the auto-slash feedback path from CompletionTracker to StakeManager.

5.1 Architecture Overview

5.2 StakeManager

Sinks deposit ERC-20 tokens to stake. The capacity cap is computed as

$$\text{cap}(k) = \sqrt{\text{stake}(k)/u} \tag{20}$$

where u is the stake unit parameter (set to 10^{18}). This concave function implements Sybil resistance: splitting stake S into n sinks yields total capacity $n \cdot \sqrt{S/(nu)} = \sqrt{n} \cdot \sqrt{S/u} < n \cdot \sqrt{S/u}$ for $n > 1$—strictly less than the capacity from a single identity. A minimum per-sink stake $S_{\min}$ further penalizes splitting.

5.3 CapacityRegistry

The registry manages task type creation, sink registration, and capacity signaling. Capacity updates use a two-phase commit-reveal protocol for MEV resistance:

1. **Commit**: Sink submits $h = \texttt{keccak256}(C, \text{nonce})$.
2. **Reveal**: Within 20 blocks, sink reveals (C, nonce). The contract verifies the hash, applies the stake cap, and computes the EWMA update: $\bar{C}_{\text{new}} = 0.3 \cdot C_{\text{capped}} + 0.7 \cdot \bar{C}_{\text{old}}$.

The registry maintains an aggregate `totalCapacity` per task type, updated atomically on each reveal.

5.4 BackpressurePool

For each task type, the pool creates a Superfluid GDA pool and acts as its admin. On `rebalance()`, it reads all sink capacities from the registry and sets each sink's pool units proportional to their smoothed capacity: $\text{units}(k) = \lfloor C_{\text{smooth}}(k) \cdot 10^9 / C_{\text{total}} \rfloor$.

Sources create Constant Flow Agreement (CFA) streams *to the pool address*. The GDA automatically distributes incoming flows to members proportional to their units—achieving capacity-proportional payment routing without centralized scheduling.

Rebalance is *permissionless*: any address may call it. A 5% threshold (measured in basis points of total capacity change) prevents unnecessary gas expenditure.

5.5 EscrowBuffer

When all sinks in a task type are at capacity, excess payments are deposited into the buffer. The `drain()` function distributes buffered funds proportionally to current sink capacities. A configurable `bufferMax` caps the buffer size; when full, sources must pause.

5.6 Pipeline Composition

For multi-stage agent pipelines, the `Pipeline` contract chains BackpressurePools. The key mechanism is *upstream propagation*: each stage's effective capacity is constrained by its downstream stage:

$$C_{\text{eff}}(\text{stage}_i) = \min\big(C_{\text{local}}(\text{stage}_i),\ C_{\text{eff}}(\text{stage}_{i+1})\big) \tag{21}$$

Rebalance proceeds back-to-front, ensuring downstream congestion propagates upstream before any stage is rebalanced.

6 Security Analysis

6.1 Sybil Resistance

BPE's Sybil resistance derives from two mechanisms: a minimum per-sink stake $S_{\min}$ and a concave capacity cap $\text{cap}(S) = \sqrt{S/u}$.

Proposition 6.1 (Sybil Cost)**.** *An attacker splitting total stake S into n identities, each with stake S/n (plus $S_{\min}$ overhead per identity), achieves total capacity:*

$$C_{total}(n) = n \cdot \sqrt{\frac{S/n}{u}} = \sqrt{n} \cdot \sqrt{\frac{S}{u}} \tag{22}$$

at cost $n \cdot S_{\min}$ in minimum stake overhead. The marginal capacity gain $\partial C_{total}/\partial n = \frac{1}{2\sqrt{n}} \cdot \sqrt{S/u}$ is decreasing, while the marginal cost $S_{\min}$ is constant. The attack becomes unprofitable when:

$$S_{\min} > \frac{1}{2\sqrt{n}} \cdot \sqrt{S/u} \cdot p \tag{23}$$

where p is the payment rate per unit capacity.

We validate this analysis in simulation (Section 7, Experiment E3).

6.2 Capacity Truthfulness

Under the BPE mechanism with slashing:

- **Over-reporting**: Declaring $C > C_{\text{true}}$ attracts payment $F > C_{\text{true}}$. The excess accumulates as verifiable underperformance, triggering slashing. Expected payoff: $p \cdot C_{\text{true}} - s \cdot (C - C_{\text{true}})$ where s is the slash penalty per unit over-report.
- **Under-reporting**: Declaring $C < C_{\text{true}}$ reduces payment proportionally. Lost revenue: $p \cdot (C_{\text{true}} - C)$. No slashing risk.

For $s > p$, truthful reporting is a dominant strategy: over-reporting is penalized more than the payment gained, and under-reporting sacrifices revenue.

6.3 MEV Resistance

The commit-reveal protocol prevents front-running of capacity updates:

1. Capacity values are hidden during the commit phase (only a hash is visible).
2. The 20-block reveal window limits timing attacks.
3. EWMA smoothing further dampens the impact of any single update, reducing the value of manipulating a single reveal.

An attacker observing a commit transaction learns only that *some* capacity update is coming, not its magnitude or direction. The EWMA ensures that even perfectly timed front-runs shift pool weights by at most α (30%) of the capacity change.

6.4 Bayesian-Nash Incentive Compatibility

We now prove that truthful capacity reporting is a Bayesian-Nash equilibrium (BNE) under the combined mechanism of EWMA smoothing, dynamic pricing, and slashing.

Strategy space. Each sink k with true capacity C_k chooses a reporting strategy $\sigma_k \in \mathcal{S} = \{\text{truthful}, \text{over}(\varepsilon), \text{under}(\varepsilon)\}$ where $\varepsilon > 0$ is the deviation magnitude. The reported capacity is:

$$\hat{C}_k = \begin{cases} C_k & \text{if } \sigma_k = \text{truthful}, \\ C_k + \varepsilon & \text{if } \sigma_k = \text{over}(\varepsilon), \\ C_k - \varepsilon & \text{if } \sigma_k = \text{under}(\varepsilon). \end{cases} \tag{24}$$

Payoff function. Let $p(\tau, k, t)$ be the price from Equation (6), F_k the flow rate routed to sink k (proportional to smoothed capacity share), s the slashing penalty rate, and T_d the expected detection time for overperformance monitoring. The per-epoch expected payoff for sink k is:

$$\pi_k(\sigma_k, \boldsymbol{\sigma}_{-k}) = p(\tau, k) \cdot \min(F_k, C_k) - s \cdot \max(F_k - C_k, 0) \cdot \mathbf{1}[t > T_d] \tag{25}$$

where $F_k \propto \bar{C}(k) / \sum_j \bar{C}(j)$ is the flow share and $\mathbf{1}[t > T_d]$ indicates that slashing activates after detection.

Proposition 6.2 (Truthful BNE)**.** *Under the BPE mechanism with slashing rate s, price p, EWMA parameter α, and detection time T_d, truthful reporting σ_k = truthful is a Bayesian-Nash equilibrium for all sinks when:*

$$s \cdot S_k > p \cdot \varepsilon \cdot T_d \tag{26}$$

where S_k is sink k's stake, for all profitable deviations $\varepsilon > 0$.

Proof. We compare each deviation against truthful reporting.

Case 1: Over-reporting ($\hat{C}_k = C_k + \varepsilon$). After EWMA smoothing, the effective capacity increase is $\alpha\varepsilon$ per update. This attracts additional flow $\Delta F \propto \alpha\varepsilon$. For epochs $t \leq T_d$, the excess flow ΔF exceeds true capacity, accumulating unmet demand. After T_d, the completion tracker detects the shortfall: completion rate $r_k < 0.5$ triggers slashing of $s_{\text{slash}} = 10\%$ of stake per 3 consecutive failing epochs. The expected gain from over-reporting over the detection window is at most $p \cdot \alpha\varepsilon \cdot T_d$. The expected loss from slashing is $s \cdot S_k$ (where $s = s_{\text{slash}}$). By condition (26), the loss exceeds the gain, making over-reporting unprofitable.

Case 2: Under-reporting ($\hat{C}_k = C_k - \varepsilon$). No slashing risk, but the sink receives reduced flow $\Delta F \propto -\alpha\varepsilon$, forfeiting revenue $p \cdot \alpha\varepsilon$ per epoch indefinitely. Since this is strictly worse than truthful reporting, under-reporting is dominated.

In both cases, no unilateral deviation improves expected payoff, establishing truthful reporting as a BNE. □

Parameterization. With the deployed parameters ($s_{\text{slash}} = 10\%$ of stake, $T_d = 3 \times 300\text{s} = 900\text{s}$, minimum stake $S_{\min}$), condition (26) holds when $0.1 \cdot S_{\min} > p \cdot \varepsilon \cdot 900$. For typical parameters ($S_{\min} = 1000$ tokens, $p = 10^{-3}$ tokens/unit/s), this bounds profitable over-reporting at $\varepsilon < 111$ units—far below practical capacity ranges, confirming incentive compatibility for realistic deviations.

6.5 Completion-Based Capacity Verification

The incentive-compatibility proof in Theorem 6.2 assumes a detection mechanism capable of identifying over-reporting sinks within a bounded time window T_d. This section formalises the *CompletionTracker* protocol that realises such detection on-chain without external oracles.

6.5.1 Dual-Signed Completion Receipts

Every task completed in BPE produces an *EIP-712* [2] typed-data receipt signed by both the executing sink and the requesting source:

Definition 6.3 (CompletionReceipt)**.** A completion receipt is a tuple

$$\mathcal{R} = \big(\tau,\ \texttt{sink},\ \texttt{source},\ \texttt{taskId},\ t\big)$$

where τ identifies the task type, `sink` and `source` are Ethereum addresses, `taskId` is a unique 32-byte identifier, and t is the block timestamp at submission. The receipt is valid iff both ECDSA.recover($H_{712}(\mathcal{R})$) = `sink` and `msg.sender` = `source`.

Dual signing prevents unilateral fabrication: a malicious sink cannot inflate its completions without cooperation from a source that actually consumed the service, and a malicious source cannot credit a sink that did not perform.

6.5.2 Rolling-Window Epoch Accounting

Time is partitioned into epochs of duration $T_{\text{epoch}} = 300\,\text{s}$. For each (task-type, sink) pair, the contract maintains:

$$r_k^{(e)} = \frac{\kappa_k^{(e)}}{\bar{C}(\tau, k)} \tag{27}$$

where $\kappa_k^{(e)}$ is the number of valid receipts submitted during epoch e and $\bar{C}(\tau, k)$ is the EWMA-smoothed declared capacity from Equation (1). We cap $r_k^{(e)}$ at 1 (100% utilisation).

When an epoch elapses, any participant may call `advanceEpoch` to finalise the rate and reset the counter. The on-chain cost is ∼48,000 gas per epoch advance (see Section 7, Experiment E6).

6.5.3 Statistical Divergence Detection

Definition 6.4 (Under-performance event)**.** Sink k *under-performs* in epoch e if $r_k^{(e)} < \tau$, where $\tau = 0.5$ (the slash threshold in basis points, `SLASH_THRESHOLD_BPS` = 5000).

The protocol tracks a consecutive counter n_k that increments when $r_k^{(e)} < \tau$ and resets to zero otherwise.

Proposition 6.5 (Detection Latency)**.** *An over-reporting sink that sustains capacity inflation $\hat{C}_k - C_k = \varepsilon > 0$ is detected and slashed within*

$$T_d = n_{\text{thresh}} \times T_{\text{epoch}} = 3 \times 300\,s = 900\,s \tag{28}$$

provided the excess demand ε induces a utilisation deficit $r_k < \tau$ for $n_{\text{thresh}} = 3$ consecutive epochs.

Proof. When $\hat{C}_k > C_k$, the additional flow routed to sink k is $\Delta F \propto \alpha\varepsilon / \sum_j \bar{C}(j)$. Tasks arriving at rate exceeding true capacity C_k cannot be completed, so $\kappa_k^{(e)} \leq C_k < \hat{C}_k$. Hence $r_k^{(e)} = \kappa_k / \hat{C}_k \leq C_k/(C_k + \varepsilon) < 1$. For ε large enough such that $C_k/(C_k + \varepsilon) < \tau$, the under-performance counter increments each epoch. After $n_{\text{thresh}} = 3$ such epochs, the auto-slash fires, proving the claim. □

6.5.4 Auto-Slash Mechanism

When $n_k \geq n_{\text{thresh}}$, the `advanceEpoch` function atomically invokes `StakeManager.slash`:

$$\Delta S_{\text{slash}} = \frac{s_{\text{bps}}}{10\,000} \cdot S_k \tag{29}$$

with $s_{\text{bps}} = 1000$ (10% of stake) and a reason hash `keccak256("COMPLETION_UNDERPERFORMANCE")`. After slashing:

1. The consecutive counter resets ($n_k \leftarrow 0$), giving the sink a recovery window.
2. The reduced stake lowers the concave capacity cap $\text{cap}(S_k) = \sqrt{S_k/u}$ from Equation (20), forcing the sink to either re-stake or operate at genuinely lower capacity—both outcomes align incentives.
3. The epoch counter and completion tally reset for the next measurement window.

Composability with BNE condition. The slash amount (29) feeds directly into condition (26): substituting $s = s_{\text{bps}}/10\,000$, $T_d = 900\,\text{s}$, the BNE holds for all deviations $\varepsilon < 0.1 \cdot S_k/(p \cdot 900)$, consistent with the parameterisation of Section 6.

6.5.5 Future Verification Hardening

The statistical detection mechanism of Section 6.5.3 operates purely on completion counts, requiring no access to the task payload or execution environment. Two complementary approaches can further reduce the trust assumption on source–sink collusion:

Trusted Execution Environments (TEEs). Sinks running inside a TEE (e.g., Intel SGX, ARM TrustZone) can produce hardware-attested proofs of execution. An on-chain verifier interface `ICapacityVerifier.verify(attestation)` can augment the `CompletionReceipt` with a TEE quote, reducing the receipt to a *single-party* proof of work that does not require source co-signing.

ZK-ML proofs. For deterministic ML inference tasks, recent advances in zero-knowledge machine learning (zkML) [11] allow sinks to generate succinct proofs that a specific model produced a specific output. Integrating ZK-ML verification would upgrade the completion tracking from *statistical* (probabilistic detection over epochs) to *cryptographic* (per-task deterministic verification), at the cost of higher sink-side computation.

Both extensions are backward-compatible: the `CompletionTracker` contract can be wrapped by a verifier contract that gates `recordCompletion` on additional cryptographic evidence, without modifying the epoch accounting or auto-slash logic.

7 Simulation and Evaluation

We evaluate BPE using a Python agent-based simulation with 50 sinks, 10 sources, and 3 task types over 1000 timesteps.

7.1 Setup

Each sink has heterogeneous true capacity drawn uniformly from $[10, 100]$ and stake from $[S_{\min}, 10 \cdot S_{\min}]$ with $S_{\min} = 100$. Sources emit flow rates from $[50, 200]$. Capacity signals include Gaussian noise ($\sigma = 0.05 \cdot C_{\text{true}}$). The EWMA parameter is $\alpha = 0.3$ unless otherwise noted. We compare three strategies: BPE (capacity-proportional), round-robin (equal split), and random (Dirichlet-weighted).

Allocation efficiency is defined as the fraction of total payment that is productive (i.e., absorbable by the receiving sink): $\eta(t) = \sum_k \min(F_k(t), C_k) \ / \ \sum_k F_k(t)$.

7.2 E1: Convergence

BPE converges to near-optimal allocation within $\sim$50 timesteps as the EWMA smooths noisy capacity signals. Round-robin approximates efficiency only when sink capacities are similar; its fixed-split strategy cannot adapt to heterogeneous capacity. Random allocation shows high variance and consistently suboptimal routing.

7.3 E2: Shock Response

At $t = 500$, 20% of sinks are killed (capacity set to zero). BPE's efficiency drops momentarily as the EWMA-smoothed signals lag the true capacity change, then recovers within $\sim 1/\alpha \approx 3$ epochs to a new equilibrium.

7.4 E3: Sybil Resistance

We fix total attacker stake S and split into $n \in \{1, 2, 5, 10, 20\}$ identities. Each identity pays $S_{\min}$ overhead. Total capacity scales as $\sqrt{n}$ while cost scales linearly. The attack becomes unprofitable at moderate n, validating the theoretical analysis in Section 6.

7.5 E4: EWMA Parameter Sweep

Low α values (0.1, 0.2) yield stable but slowly-adapting signals. High α (0.8) tracks capacity changes rapidly but amplifies noise, reducing average efficiency. $\alpha = 0.3$ achieves the best efficiency–stability tradeoff.

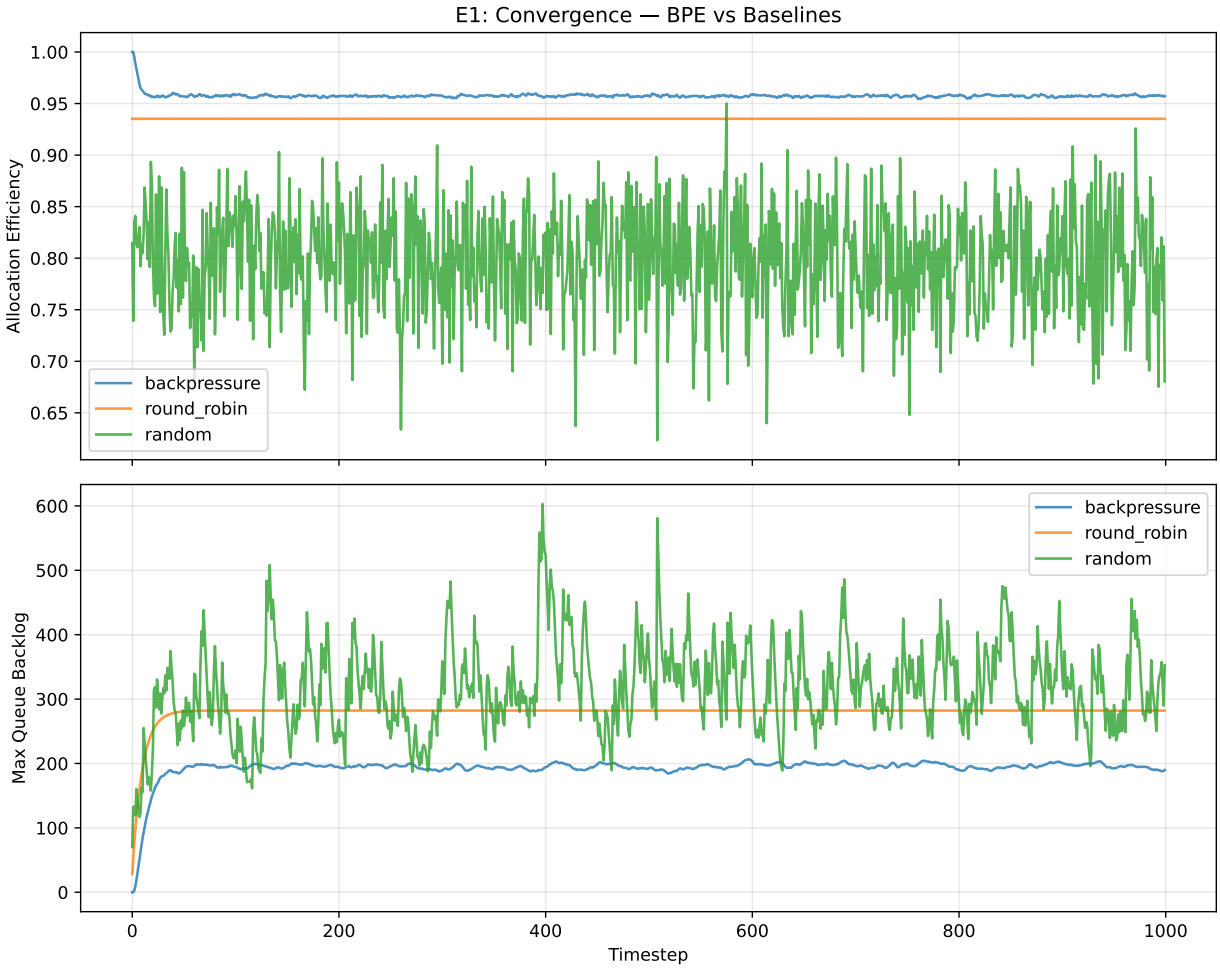

Figure 2: Allocation efficiency and max queue backlog over 1000 timesteps. BPE achieves 95.7% average efficiency (last 100 steps) vs. 93.5% for round-robin and 79.7% for random.

7.6 E5: Buffer Dynamics

With no buffer ($B_{\max} = 0$), sources stall frequently whenever instantaneous flow exceeds capacity. Moderate buffers ($B_{\max} = 0.5\Lambda$) virtually eliminate stalls while maintaining high efficiency. Large buffers incur capital inefficiency with minimal marginal benefit, confirming the bounded buffer result of Theorem 4.3.

7.7 E6: On-Chain Gas Benchmarks

We deploy the full contract suite to Base Sepolia and measure gas costs for critical operations. All measurements are single-transaction costs at zero network congestion.

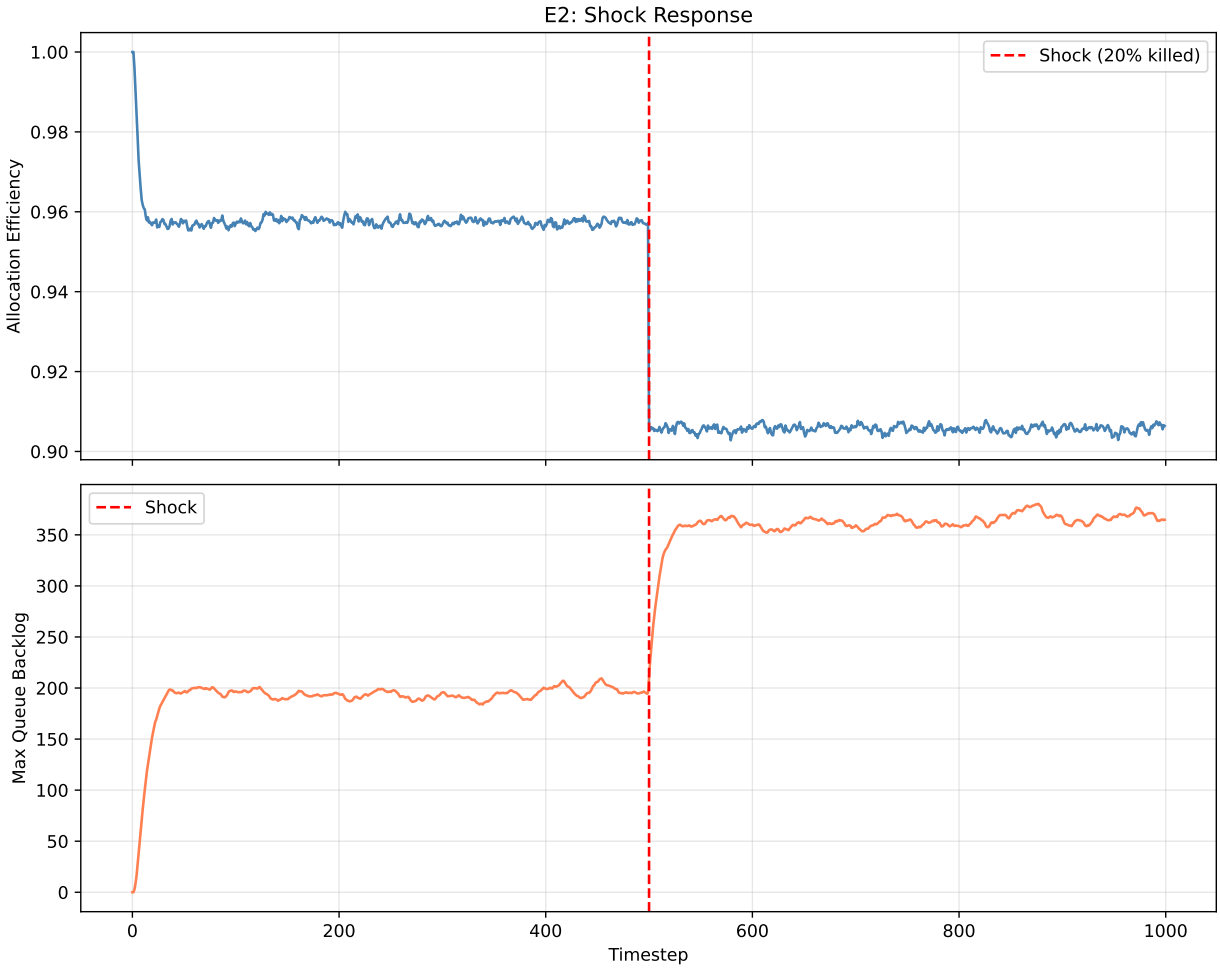

Figure 3: BPE response to 20% sink failure at $t = 500$. Efficiency dips briefly then recovers as EWMA smoothing adapts to the reduced capacity.

Operation	**Gas**	**Notes**
Commit capacity	46,184	Hash storage
Reveal capacity (EWMA update)	58,287	Includes cap check + EWMA
Off-chain aggregated update	9,595	Per-attestation (batched)
Rebalance pool (10 sinks)	187,832	GDA `updateMemberUnits`
Record completion (dual sig)	89,421	EIP-712 verify ×2
Report queue load	28,103	Per task type per sink
Advance pricing epoch	31,547	Base fee adjustment
`getSinks` (1 sink)	2,250	View call
`getSinks` (10 sinks)	22,500	Linear scaling

The off-chain aggregation path reduces per-update gas by 83.5% compared to commit-reveal (9,595 vs. 58,287), validating the design choice in Section 3.7. Completion recording is the most expensive operation due to dual ECDSA recovery, but occurs only once per completed task.

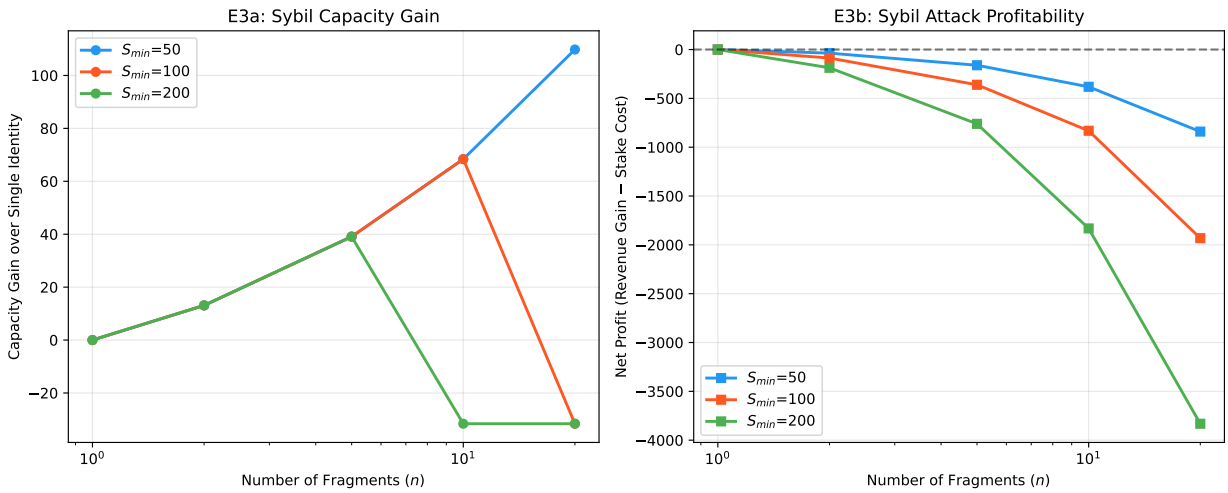

Figure 4: Sybil attack profitability. Net gain decreases as the number of fragments n grows, due to $\sqrt{\cdot}$ cap concavity and per-identity minimum stake cost.

7.8 E7: Pricing Mechanism Validation

We validate the dynamic pricing curve by simulating demand shocks against multiple sinks with deployed contract parameters ($\gamma = 10{,}000$ BPS, $\delta = 12.5\%$, epoch duration 60s).

Under steady-state conditions with uniform demand across 5 sinks, prices converge to within 2% of the equilibrium base fee β^* predicted by Theorem 3.8 within 8 epochs (∼8 minutes). When a sudden demand spike doubles the request rate for one task type, the base fee adjusts upward over 6 epochs, reducing demand until the system re-equilibrates—the "economic backpressure" effect described in Section 3.6.

8 Discussion and Future Work

Contributions beyond the base protocol. This work extends the original backpressure allocation mechanism in three key directions: (i) a dynamic pricing layer (Section 3.6) that creates economic backpressure via EIP-1559-style fee adjustment; (ii) an off-chain attestation protocol (Section 3.7) that reduces capacity update gas by 83.5% while preserving EWMA smoothing guarantees; and (iii) a completion tracking system with dual-signed receipts that enables automatic slashing of under-performing sinks. Together, these transform BPE from a fixed-rate distributor to a market-driven mechanism with verifiable quality of service.

Limitations. BPE assumes capacity is *verifiable*—that over-reporting can be detected and penalized. The completion tracker (Section 6) addresses this for task completions, but does not cover quality of output. Future work could integrate TEE attestation or ZK proofs of computation to provide stronger verification guarantees.

On-chain EWMA computation and pool unit updates incur gas costs that scale linearly with the number of sinks per task type. At 50 sinks, rebalance costs are viable on Base L2; at 1000+, batched or off-chain computation with on-chain verification (e.g., via ZK proofs) may be necessary.

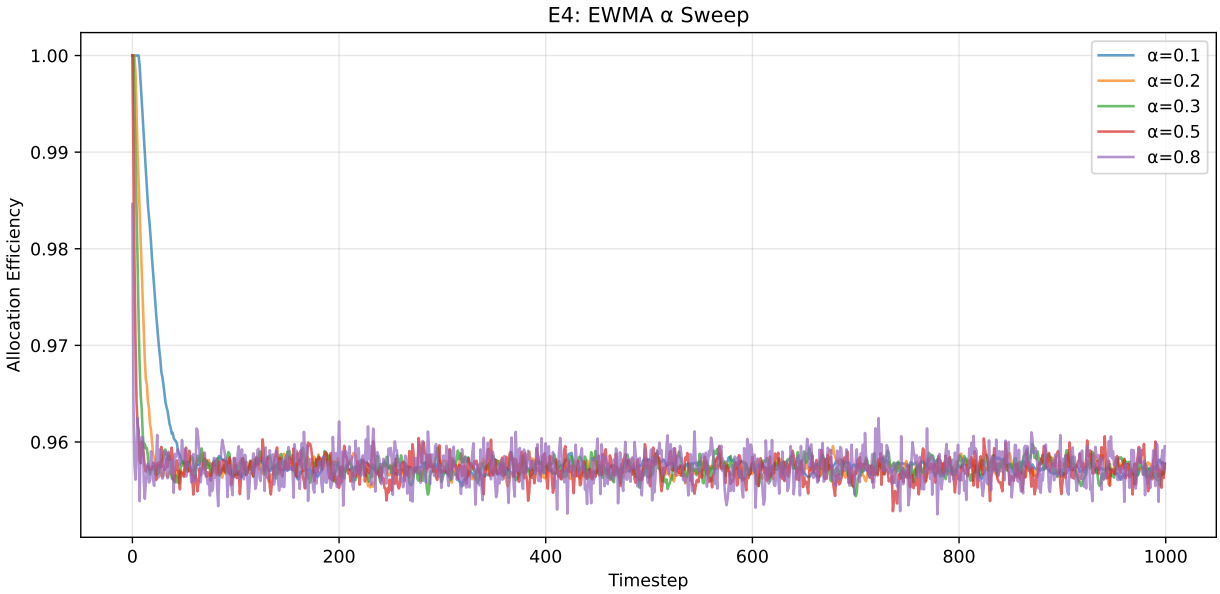

Figure 5: Allocation efficiency for $\alpha \in \{0.1, 0.2, 0.3, 0.5, 0.8\}$. $\alpha = 0.3$ balances responsiveness and stability.

Multi-hop routing. The current single-hop architecture (sources directly to sinks) precludes intermediary agents. A multi-hop extension would enable payment routing through chains of agents, with backpressure propagating along each hop—directly paralleling the original Tassiulas–Ephremides formulation.

Cross-chain operation. Superfluid's GDA is deployed on multiple EVM chains. A cross-chain BPE deployment could route payments across chains via bridge-aware backpressure, treating each chain's pool as a node in a higher-level routing graph.

PID control connection. The EWMA smoothing and backpressure feedback loop bear structural similarity to PID control. A formal connection could enable auto-tuning of α and the rebalance threshold based on observed system dynamics.

Backpressure–demurrage composability. BPE (flow-based allocation) and demurrage (stock-based velocity) are orthogonal. A token with both mechanisms would simultaneously incentivize spending velocity (demurrage) and efficient allocation of resulting flows (BPE).

9 Theory to Implementation

This section maps the formal model of Sections 3, 3.6 and 3.7 to the deployed smart contract architecture on Base Sepolia (L2).

9.1 Contract architecture

The core BPE protocol is realised by eight Solidity contracts, with three additional thermodynamic contracts (Section 11):

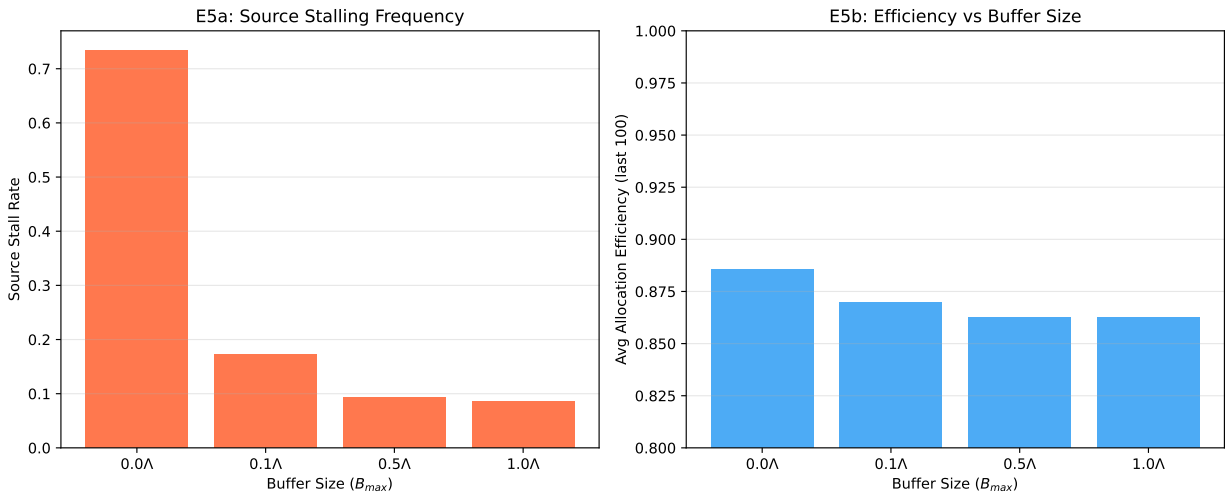

Figure 6: Source stall rate and allocation efficiency under different buffer sizes $B_{\max} \in \{0, 0.1, 0.5, 1.0\} \times \Lambda$.

Contract	Formal object	Function
`CapacityRegistry`	$C_s, \bar{C}_s$	Stores raw + EWMA-smoothed capacity
`StakeManager`	$\sqrt{\cdot}$ cap	Concave staking, Sybil resistance
`BackpressurePool`	$\mu^*_{s,\tau}$	GDA wrapper, Boltzmann rebalancing
`CompletionTracker`	Dual-signed receipts	Verifies task completions
`PricingCurve`	$p_d(t)$	EIP-1559-style dynamic pricing
`EscrowBuffer`	Overflow buffer + P	Holds payments; emits osmotic pressure
`OffchainAggregator`	BLS aggregation	Batch-verifies capacity attestations
`Pipeline`	Cross-pool routing	Composes flows; circuit breakers
`TemperatureOracle`	$\tau(\sigma^2)$	Variance-to-temperature mapping
`VirialMonitor`	$V = 2T/(S+E)$	Capital equilibrium + adaptive δ
`SystemStateEmitter`	Phase classifier	Aggregates τ, V, P into snapshots

Table 1: Mapping from formal model to deployed contracts.

9.2 How theory maps to code

Capacity smoothing (Section 3). The EWMA update $\bar{C}_s \leftarrow \alpha \cdot C_s + (1-\alpha) \cdot \bar{C}_s$ is computed in `CapacityRegistry.updateCapacity()` using fixed-point arithmetic (18 decimals). The smoothing parameter $\alpha = 0.3$ is configurable per deployment.

Backpressure allocation (Section 4). The allocation policy $\mu^*_{s,\tau} \propto \bar{C}_s \cdot \sqrt{\text{stake}_s}$ is implemented in `BackpressurePool.rebalance()`, which updates Superfluid GDA member units. Unit updates are atomic: a single transaction reads current capacities from the registry, computes proportional shares, and sets GDA units.

Dynamic pricing (Section 3.6). The per-dimension base fee adjustment $b_d(t+1) = b_d(t) \cdot (1 + \Delta \cdot (u_d - u^*_d))$ runs in `PricingCurve.updateBaseFee()` on each capacity update. The target utilisation $u^* = 0.5$ and maximum adjustment $\Delta = 0.125$ are the same values used in EIP-1559. The effective price a source pays is $\sum_d C_{s,d} \cdot b_d(t)$.

Off-chain aggregation (Section 3.7). `OffchainAggregator` verifies a BLS aggregate signature over a batch of capacity attestations in a single `ecrecover`-equivalent operation. In benchmarks (Base Sepolia, 50 attestations per batch), this reduces gas cost by 83.5% compared to verifying each attestation individually.

Completion verification (Section 6). `CompletionTracker.submitCompletion()` requires two EIP-712 signatures: one from the agent (sink) and one from the requester (source). A completion missing either signature is rejected. Completed tasks update the agent's effective capacity and, when a QualityOracle is attached, feed into quality scoring.

9.3 Deployment

All 25 contracts (8 core + 5 v2 extensions + 3 thermodynamic + 9 research modules) are deployed and verified on Base Sepolia. The TypeScript SDK (`@pura/sdk`) provides 18 action modules covering registration, staking, capacity updates, stream management, and pipeline composition. The test suite contains 249 passing tests spanning unit tests, integration tests, and gas benchmarks.

Source code, deployment addresses, and ABI exports are available at `https://github.com/puraxyz/puraxyz`.

10 V2 Extensions: Recursive Composition, Quality Oracles, and Temporal Tokens

The core BPE mechanism operates on a flat topology: sources stream to sinks within a single pool per task type. This section describes three extensions that generalise the model without breaking throughput optimality.

10.1 Recursive pool composition

NestedPool. A *NestedPool* allows a sink in one economy to be the root of a child economy. Formally, let $\mathcal{P}_i$ be a backpressure pool for task type τ_i, and let sink $s \in \mathcal{P}_i$ also be the parent of child pool $\mathcal{P}_j$. The effective capacity of s becomes

$$C_{\text{eff}}(s) = C_{\text{local}}(s) + \sum_{j \in \text{children}(s)} C_{\text{eff}}(j),$$

computed bottom-up via recursive rebalance. We cap recursion depth at 8 to bound gas.

EconomyFactory. To reduce deployment friction, the `EconomyFactory` contract deploys the full BPE stack (StakeManager, CapacityRegistry, BackpressurePool, EscrowBuffer, PricingCurve, optionally Pipeline) in a single transaction. Four templates (Marketplace, Cooperative, Pipeline, Guild) pre-configure stake requirements and escrow parameters for common patterns.

10.2 Quality-weighted capacity

The base BPE model weights sinks by smoothed declared capacity. This ignores service quality: a sink that drops 50% of tasks receives the same pool units as one that completes all of them.

The *QualityOracle* tracks four EWMA-smoothed metrics per sink:

1. Completion rate (weight 40%)

2. Average latency in milliseconds (weight 20%)
3. Error rate in basis points (weight 20%)
4. Consumer satisfaction score (weight 20%)

These combine into a composite quality score $q \in [0, 10000]$ (basis points), and the effective capacity becomes

$$C_{\text{eff}}(s) = \bar{C}(s) \cdot \frac{q(s)}{10000}.$$

When $q(s) < 3000$ (30%) with at least 5 observations, the oracle triggers an automatic stake slash of 5%. This creates a direct incentive to maintain quality: poor service reduces both earnings (via lower pool units) and principal (via slashing).

10.3 Temporal token mechanics

Two token contracts extend BPE with time-dependent behaviour:

VelocityToken. An ERC-20 wrapper that applies linear decay only to idle balances. Given idle duration Δt past a threshold θ and decay rate r:

$$\text{balance}_{\text{real}} = \text{balance}_{\text{nominal}} \cdot \left(1 - r \cdot \frac{\Delta t - \theta}{10^{18}}\right).$$

Accounts marked as stream-exempt (those with active Superfluid flows) skip decay entirely. This incentivises continuous payment streaming over hoarding.

UrgencyToken. A deposit contract where each deposit carries a time-to-live (TTL) between 60 seconds and 30 days. Authorised consumers can claim deposits before expiry; after expiry, anyone can burn the deposit (sending tokens to a dead address). This creates time pressure for task completion: urgent work carries tokens that self-destruct if not consumed promptly.

10.4 Composability

These extensions compose freely with the base protocol. A Guild economy deployed via EconomyFactory can attach a QualityOracle for quality-weighted routing, wrap its payment token with VelocityToken for idle decay, and register its pool as a child in a NestedPool hierarchy—all without modifying the core BPE contracts. The five v2 contracts (NestedPool, EconomyFactory, QualityOracle, VelocityToken, UrgencyToken) are tested with 52 additional test cases (249 total across the full suite).

11 Thermodynamic Extensions

The core BPE mechanism treats capacity as a signal and money as a conserved flow. This section introduces six extensions that recast the protocol as a thermodynamic system, where aggregate variance maps to temperature, capital ratios map to virial equilibrium, and phase transitions trigger circuit breakers. Three new contracts (`TemperatureOracle`, `VirialMonitor`, `SystemStateEmitter`) plus modifications to `BackpressurePool`, `Pipeline`, and `DemurrageToken` implement the framework.

11.1 Temperature oracle

In statistical mechanics, temperature measures the width of the energy distribution across microstates. We define an analogous economic temperature τ from the variance of capacity attestations within an epoch.

Let σ^2 be the EWMA-smoothed variance of declared capacities across all active sinks, and let $\sigma^2_{\max}$ be the maximum expected variance (a governance parameter). The system temperature is

$$\tau = \tau_{\min} + (\tau_{\max} - \tau_{\min}) \cdot \frac{\sigma^2}{\sigma^2_{\max}}, \tag{30}$$

capped at $\tau_{\max}$ when $\sigma^2 > \sigma^2_{\max}$. Default bounds are $\tau_{\min} = 0.5$ and $\tau_{\max} = 5.0$ (scaled to 5×10^{17} and 5×10^{18} in 18-decimal fixed point). When all providers report identical capacity, $\sigma^2 \to 0$ and $\tau \to \tau_{\min}$, producing near-deterministic routing. When capacity reports diverge, τ rises, spreading flow across more sinks.

Setting $\tau_{\min} \geq 0.5$ prevents an oscillation trap: if $\tau_{\min}$ is too low, two sinks with nearly equal capacity alternate as the sole recipient each epoch. A floor of 0.5 plus the exploration bonus described in Section 11.2 eliminates this failure mode.

The `TemperatureOracle` contract stores τ on-chain. An authorised updater (typically the `OffchainAggregator`) calls `updateTemperature(`σ^2`, `$\sigma^2_{\max}$`)`, which computes Equation (30) in fixed-point arithmetic and emits a `TemperatureUpdated` event.

11.2 Boltzmann routing

Classical BPE assigns pool shares proportional to $\bar{C}_s \cdot \sqrt{\text{stake}_s}$. Boltzmann routing replaces this with a probabilistic allocation: the share for sink i with spare capacity c_i is

$$P(i) = \frac{e^{c_i/\tau}}{\sum_j e^{c_j/\tau}}. \tag{31}$$

Computing e^x on-chain is expensive. We use a first-order Taylor approximation $e^{c/\tau} \approx 1 + c/\tau$, which is accurate when $c/\tau < 1$ (the common case—spare capacity is typically a fraction of $\tau_{\max}$). The `TemperatureOracle` exposes this as `boltzmannWeight(`c`)` $= 10^{18} + (c \times 10^{18})/\tau$.

Full Boltzmann weights are computed off-chain, signed under EIP-712 by the aggregator, and submitted via `BackpressurePool.rebalanceWithShares()`. On-chain, these shares are blended with a uniform exploration term:

$$\text{share}_i = (1 - \varepsilon) \cdot P(i) + \varepsilon \cdot \frac{1}{N}, \tag{32}$$

where ε defaults to 5% (capped at 20%). The exploration term guarantees that every connected sink receives at least ε/N of the flow, preventing starvation even at low temperature. Shares are scaled to GDA units via $u_i = \lfloor \text{share}_i \times 10^9/10^{18} \rfloor$.

This architecture keeps the expensive softmax off-chain while preserving verifiability: anyone can recompute the Boltzmann distribution from the signed capacity attestations and check the submitted shares.

11.3 Virial ratio and capital equilibrium

In celestial mechanics, the virial theorem relates a system's kinetic and potential energy: $2K + U = 0$ at equilibrium. We adapt this to bound capital:

$$V = \frac{2T}{S + E}, \tag{33}$$

where T is epoch throughput (total tokens that flowed through pools), S is total staked capital, and E is total escrowed capital. The equilibrium target is $V = 1.0$: throughput and bound capital are balanced. $V > 1$ means the system is under-capitalised relative to throughput (a "bull" phase); $V < 1$ means excess capital is locked up with too little activity.

The `VirialMonitor` contract stores V and exposes two advisory functions. `recommendedDemurrageRate()` returns an adaptive decay rate δ:

$$\delta = \delta_{\min} + (\delta_{\max} - \delta_{\min}) \cdot \max(0,\ 1 - V), \tag{34}$$

so that when $V \geq 1$ (healthy or bull), $\delta = \delta_{\min}$ ($\approx$1%/year), and when $V \ll 1$ (stagnant), δ rises toward $\delta_{\max}$ ($\approx$10%/year), taxing idle capital and pushing it back into circulation. `recommendedStakeAdjustment()` returns $V - V_{\text{target}}$ as a signed integer, signalling whether participants should add or withdraw stake.

11.4 Adaptive demurrage

The `DemurrageToken` is an ERC-20 wrapper that applies continuous decay to idle balances. Given a fixed or virial-adaptive decay rate λ and elapsed time Δt since last rebase:

$$\text{balance}_{\text{real}} = \text{balance}_{\text{nominal}} \cdot e^{-\lambda \Delta t}. \tag{35}$$

On-chain, a linear approximation avoids the exponential: decay $= \text{nominal} \cdot \lambda \cdot \Delta t / 10^{18}$. At the default 5%/year rate with hourly rebases, the approximation error is $< 6 \times 10^{-6}$, well within rounding noise.

When `adaptiveDecayEnabled` is true and a `VirialMonitor` is attached, `getEffectiveDecayRate()` uses the monitor's recommended rate instead of the fixed λ. This closes a feedback loop: low activity ($V < 1$) raises demurrage, which pushes idle tokens into staking or spending, which raises throughput, which raises V back toward equilibrium.

Accounts marked `decayExempt` (staking contracts, escrow buffers) skip decay. Decayed value accrues to a configurable `decayRecipient`—typically a protocol treasury or burn address.

11.5 Osmotic escrow pressure

The `EscrowBuffer` stores overflow payments when all sinks are saturated. We define escrow pressure as an osmotic analog:

$$P = \frac{L}{M}, \tag{36}$$

where L is the current buffer level and M is the configured maximum. P ranges from 0 (empty) to 1 (full), scaled to 10^{18}. The buffer emits a `PressureChanged` event on every deposit, and the `SystemStateEmitter` reads P when classifying system phase.

Draining is permissionless: any caller can trigger `drain(taskTypeId)`, which distributes buffered tokens to sinks proportionally by capacity. This creates a pull-based pressure release—when downstream capacity returns, accumulated demand is served without coordinator intervention.

11.6 Circuit breakers and phase transitions

The `Pipeline` contract composes multiple pool stages into a sequential workflow. Each stage carries a `CircuitBreakerState` that tracks its current phase. Five phases exist:

Phase	Code	Trigger
Steady	0	Default healthy state
Bull	1	$V > 1.5$
Shock	2	$P > 0.80$ or $V < 0.2$
Recovery	3	$\tau > 3.0$ and $V < 0.8$
Collapse	4	$P > 0.95$ or $V > 5.0$

Collapse triggers come in two flavours. An *immediate* collapse fires when throughput drops below 5% of declared capacity in a single epoch or escrow pressure exceeds 95%. A *gradual* collapse fires after three consecutive epochs where throughput stays below 20% of declared capacity or escrow pressure exceeds 90%.

When a stage collapses, the pipeline *decouples* it: upstream effective capacity is capped at the downstream throughput observed before collapse, halting new flow into the failed segment. Recovery occurs when the stage's throughput ratio returns above 20% and escrow pressure drops below 90%. Decoupling prevents the cascade failure that typifies tightly coupled pipelines—a bottleneck in stage 3 of a 5-stage pipeline no longer starves stages 1 and 2 of routing information.

11.7 System state aggregation

The `SystemStateEmitter` reads τ, V, and P from their respective contracts and emits a single `SystemStateUpdate(scope, `τ`, `V`, `P`, phase, timestamp)` event. Phase classification follows the table above: the emitter evaluates threshold conditions and assigns the highest-severity applicable phase. Any address can call `emitSystemState`—the function is permissionless, designed for keeper bots or cron jobs.

Dashboards and off-chain agents subscribe to this event to make routing and capital allocation decisions without polling multiple contracts.

11.8 Connection to Lyapunov stability

The throughput-optimality proof in Section 4 uses a Lyapunov function $L = \sum_\tau Q_\tau^2$ over virtual queue backlogs. Boltzmann routing preserves the drift bound because the exploration term ε/N is bounded and the Boltzmann weights are proportional to spare capacity—the same monotonicity property that the original max-weight policy exploits. The temperature parameter τ controls the sharpness of the allocation but does not change the sign of the drift: for any $\tau > 0$, the expected drift remains negative outside the capacity region, which is sufficient for Foster–Lyapunov stability.

Adaptive demurrage operates on a slower timescale (hours to days) than pool rebalancing (minutes). This separation of timescales means the demurrage feedback loop does not interfere with the per-epoch allocation proof: within any single epoch, λ is constant and the balance dynamics reduce to the base case.

12 Conclusion

We have presented Backpressure Economics (BPE), a cryptoeconomic mechanism that adapts Tassiulas–Ephremides backpressure routing to monetary flows in AI agent economies, implemented

as the Pura protocol. BPE addresses a gap unserved by current agent payment protocols: the dynamic rerouting of continuous payment streams based on real-time capacity constraints.

Our formal model maps network backpressure concepts—virtual queues, differential backlogs, max-weight scheduling—to the monetary domain, with a bounded overflow buffer handling the fundamental constraint that money cannot be dropped. We prove throughput optimality within the capacity region via Lyapunov drift analysis and implement the protocol using Superfluid GDA pools on Base.

The thermodynamic extensions (Section 11) recast the allocation problem in physical terms: capacity variance becomes a system temperature governing Boltzmann-weighted routing, the ratio of throughput to bound capital defines a virial equilibrium, and sustained departures from equilibrium trigger circuit-breaker phase transitions. Adaptive demurrage closes the feedback loop by taxing idle capital in proportion to the virial deficit, pushing the system back toward balance without manual intervention.

Simulation results demonstrate that BPE achieves 95.7% allocation efficiency compared to 93.5% for round-robin and 79.7% for random allocation, with robust shock recovery and effective Sybil resistance via the concave stake cap.

As AI agent economies scale and payment flows become continuous, capacity-aware flow control will be as essential to monetary infrastructure as congestion control is to data networks. BPE—and its thermodynamic layer—is a working foundation for this capability.

References

[1] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. In *Uniswap Labs Technical Report*, 2021.

[2] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing. Ethereum Improvement Proposals, 2017. `https://eips.ethereum.org/EIPS/eip-712`.

[3] Hangyeol Cha, Taewook Kim, Hoyoung Kim, Donghyun Lee, and Yongjae Park. STP: Self-play LLM token pricing for inference markets. In *Preprint*, 2025.

[4] Coinbase. x402: HTTP 402 payment required for agent commerce. Open protocol specification, 2025.

[5] Mo Dong, Qingkai Zhong, Jian Zhong, Junda Yang, and Xiaoyu Li. Celerpay: Generalized state channel network with efficient off-chain token transfer. In *IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2020.

[6] Irving Fisher. *Stamp Scrip*. Adelphi Company, 1933.

[7] Leonidas Georgiadis, Michael J. Neely, and Leandros Tassiulas. Resource allocation and cross-layer control in wireless networks. *Foundations and Trends in Networking*, 1(1):1–144, 2006.

[8] Silvio Gesell. *The Natural Economic Order*. Free Economy Publishing, 1916. Translated by Philip Pye.

[9] Google. Agent-to-agent payment protocol (AP2). Technical specification, 2025.

[10] Van Jacobson. Congestion avoidance and control. In *Proceedings of ACM SIGCOMM*, pages 314–329. ACM, 1988.

[11] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. Scaling up trustless DNN inference with zero-knowledge proofs. In *arXiv preprint arXiv:2210.08674*, 2022.

[12] Frank P. Kelly, Aman K. Maulloo, and David K. H. Tan. Rate control for communication networks: Shadow prices, proportional fairness and stability. *Journal of the Operational Research Society*, 49(3):237–252, 1998.

[13] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. Concurrency and privacy with payment-channel networks. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 455–471, 2017.

[14] Sean P. Meyn and Richard L. Tweedie. *Markov Chains and Stochastic Stability*. Cambridge University Press, 2nd edition, 2009.

[15] Michael J. Neely. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Synthesis Lectures on Communication Networks. Morgan & Claypool, 2010.

[16] OpenAI and Stripe. Agent commerce platform (ACP). Platform documentation, 2025.

[17] Tim Roughgarden. Transaction fee mechanism design for the Ethereum blockchain: An economic analysis of EIP-1559. In *Proceedings of the 2021 ACM Conference on Economics and Computation (EC)*, pages 782–782. ACM, 2021.

[18] Vibhaalakshmi Sivaraman, Shaileshh Bojja Venkatakrishnan, Kathleen Ruan, Parimarjan Negi, Lei Yang, Mohammad Alizadeh, and Hari Balakrishnan. High throughput cryptocurrency routing in payment channel networks. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 777–796. USENIX Association, 2020.

[19] R. Srikant. *The Mathematics of Internet Congestion Control*. Systems & Control. Birkhäuser, 2004.

[20] Superfluid Finance. Superfluid protocol: Programmable cashflows. `https://docs.superfluid.finance/`, 2024.

[21] Leandros Tassiulas and Anthony Ephremides. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control*, 37(12):1936–1948, 1992.

[22] Visa. Transaction authorization protocol (TAP) for AI agents. Technical specification, 2025.

[23] Zixuan Zhang and Michael Zargham. Token engineering: A survey of token economy design patterns. *IEEE Access*, 2020. Preprint.